
EV Vehicle Control Unit Proof of Concept

Telemetry, Drive Control and Sensor Interfacing
on the STM32F446RE

*A bench-scale hardware-in-the-loop model of an electric-vehicle
VCU: PRND gear logic, drive modes, throttle-by-wire, thermal protection,
Coulomb-counting state of charge, and speed telemetry.*

Target MCU	STM32F446RET6 (NUCLEO-F446RE)
Toolchain	STM32CubeIDE, HAL, CubeMX 6.8.1
Firmware package	STM32Cube FW_F4 V1.27.1
Telemetry host	Raspberry Pi (UART, 115 200 baud)
Document date	July 2, 2026

Contents

How to Read This Document	2
1 Nomenclature and Variable Lookup Table	3
1.1 Runtime variables	3
1.2 Compile-time constants	4
1.3 Peripheral allocation at a glance	5
2 System Overview	6
2.1 The driver's view — user interface	6
2.2 The engineer's view — hardware block diagram	6
3 Hardware Modules	9
3.1 STM32F446RE — the VCU brain	9
3.2 ACS712-05B current sensor	9
3.3 KY-013 analog temperature sensor	9
3.4 L298N motor driver	9
3.5 Johnson 12 V DC motor	10
3.6 Slotted optical speed sensor (LM393 module)	10
4 Interfacing: Connections and Pseudo Code	11
4.1 Accelerator pedal	11
4.2 Brake pedal	12
4.3 Current sensor (ACS712-05B)	12
4.4 Analog temperature sensor (KY-013)	14
4.5 Speed sensor	14
4.6 Switch debouncing	15
5 Control Logic	17
5.1 The 100 Hz VCU loop	17
5.2 Safety priority chain	17
5.3 Gear FSM (PRND)	17
5.4 Motor output stage	18
5.5 State of charge (Coulomb counting)	18
5.6 Turn signals	18
6 Telemetry and HMI	19
6.1 UART Link to Raspberry Pi	19
6.2 Telemetry Frame Format	19
6.3 Raspberry Pi Receiver	19
6.4 HMI Dashboard	19
6.5 Status and Open Items	19
A Complete Pin Map (from the CubeMX .ioc)	20

How to Read This Document

This report documents the current state of a bench-scale electric-vehicle Vehicle Control Unit (VCU) built around an STM32F446RE microcontroller. It is written to be read front to back, like a book, but each chapter is also self-contained enough to be used as a reference.

- **Chapter 1** is the lookup table: every firmware variable and constant, what it means physically, and which pin or peripheral it maps to. When any later chapter mentions a symbol such as `motor_current_amps`, this is where its full definition lives.
- **Chapter 2** shows the system from two altitudes: what the *driver* sees (user interface), and what the *engineer* sees (hardware block diagram).
- **Chapter 3** gives a short, datasheet-grounded brief on every hardware module used.
- **Chapter 4** is the core of the document: for each sensor and input, the actual electrical connection to the MCU, the reasoning behind it, and pseudo code matching the implemented firmware.
- **Chapter 5** describes the control logic that consumes those inputs: the gear FSM, drive modes, the safety priority chain, and Coulomb-counting state of charge.
- **Chapter 6** covers the UART link to the Raspberry Pi and the HMI dashboard.
- **Appendix A** is the complete pin map extracted from the CubeMX `.ioc` file.

Chapter 1

Nomenclature and Variable Lookup Table

This chapter is the single source of truth for every symbol used in the firmware and in this report. Each entry states what the variable *is* in software, what physical quantity it *represents*, and what it *maps to* on the hardware.

1.1 Runtime variables

Variable	Represents	Hardware mapping
raw_throttle_adc	Raw 12-bit conversion of the accelerator-pedal position, 0–4095.	Potentiometer wiper on PA0 (label acc_pedal), ADC1 channel 0, rank 1 of the scan sequence.
throttle_percent	Commanded PWM duty cycle, 0–99.	Written to TIM2 CCR1; drives L298N ENA through PA15.
raw_brake_adc	Raw 12-bit conversion of the brake-pedal position.	Potentiometer on PC2 (label break_pedal), ADC3 channel 12.
brake_percent	Brake pedal travel, 0–100 %.	Derived from raw_brake_adc; above 5 % it overrides throttle.
raw_current_adc	Raw 12-bit conversion of the ACS712 output after the voltage divider.	ACS712-05B VIOUT → resistive divider → PA1 (label Curren_t_sensor), ADC2 channel 1.
motor_current_amps	Instantaneous motor/battery current in amperes (signed).	Computed from raw_current_adc, ZERO_OFFSET and COUNTS_PER_AMP.
capacity_consumed_ah	Charge drawn from the battery since power-up, in ampere-hours.	Software integrator (Coulomb counter); no direct pin.
current_soc_percent	Battery state of charge, 0–100 %.	Derived from capacity_consumed_ah against battery_capacity_ah.
battery_capacity_ah	Modelled battery capacity (4 Ah at 12 V, i.e. 48 Wh).	Constant describing the bench battery.
raw_temp_adc	Raw 12-bit conversion of the NTC divider node.	KY-013 analog output on PC1 (label temp_motor), ADC1 channel 11, rank 2.
motor_temp_C	Motor temperature in °C.	Computed from raw_temp_adc via the Beta equation (Section 4.4).
ir_pulse_count	Speed-sensor pulses accumulated in the current RPM window. volatile; written in the EXTI ISR, read in the main loop.	Slotted optical sensor on PC0 (label Speed_sensor), EXTI line 0.
motor_rpm	Motor shaft speed in revolutions per minute.	Computed once per second from ir_pulse_count.
gear_state	Current gear of the FSM: PARK, NEUTRAL, DRIVE, REVERSE.	Selected by push buttons PB1 (P), PA9 (N), PB7 (D), PB2 (R); acted on through L298N IN1/IN2.

Variable	Represents	Hardware mapping
current_drive_mode	Active throttle map: ECO, CITY (default), SPORT.	Buttons PB10 (Eco), PB4 (City), PB5 (Sport).
blinker_state	Turn-signal state: OFF, LEFT, RIGHT.	Inputs PB12/PB13; LED outputs PB14/PB15.
warning_blinks_remaining	Remaining LED toggles of the “blocked shift” warning sequence.	On-board LED LD2 on PA5.
last_vcu_tick	SysTick timestamp of the last 10 ms control-loop execution.	SysTick, 1 ms resolution.
last_rpm_tick	SysTick timestamp of the last RPM window rollover.	SysTick.

1.2 Compile-time constants

Constant	Value	Meaning / justification
VCU_LOOP_MS	10	Control-loop period: 100 Hz. Fast enough for pedal feel; slow enough that polled ADC reads are negligible load.
DEBOUNCE_MS	30	A button must read pressed continuously for 30 ms before an event fires. Typical tactile-switch bounce is under 10 ms; 30 adds margin.
ZERO_OFFSET	2100	ADC count corresponding to 0 A at the divided ACS712 output (Section 4.3).
COUNTS_PER_AMP	152	ADC counts per ampere after divider attenuation (Section 4.3).
Current deadband	± 0.20 A	Readings inside this band are forced to zero to reject Hall-sensor noise and offset error.
NTC_SERIES_R	10 k Ω	Fixed series resistor of the thermistor divider.
NTC_NOMINAL_R	10 k Ω	NTC resistance at 25 °C (T_0).
NTC_NOMINAL_TEMP_K	298.15	T_0 in kelvin.
NTC_BETA	3950	Beta coefficient of the NTC.
TEMP_WARNING_C	50 °C	Above this, throttle is soft-capped at 50 %.
TEMP_FAULT_C	65 °C	Above this, throttle is cut to zero (full derate).
PULSES_PER_REV	1	The encoder disc has one slot; one rising edge per revolution.
RPM_WINDOW_MS	1000	RPM is recomputed every second; a long window improves low-RPM resolution.
RPM_GUARD_THRESHOLD	150	N→R is blocked above this RPM (calibration pending).
BLINKER_DURATION_MS	3000	Turn signals auto-cancel after 3 s.
BLINK_INTERVAL_MS	250	Warning-LED toggle period; 6 toggles = 3 flashes = 1.5 s.
TIM2 Prescaler	833	84 MHz/834 $\approx$ 100.7 kHz timer clock.
TIM2 Period (ARR)	99	100 counts per PWM period $\Rightarrow \approx 1.007$ kHz PWM, duty resolution of 1 %.

1.3 Peripheral allocation at a glance

Peripheral	Channel/Line	Pin	Signal
ADC1	IN0 (rank 1)	PA0	Accelerator pedal
ADC1	IN11 (rank 2)	PC1	Motor temperature (NTC)
ADC2	IN1	PA1	Motor current (ACS712)
ADC3	IN12	PC2	Brake pedal
TIM2	CH1 (PWM)	PA15	L298N ENA (speed)
EXTI	Line 0	PC0	Speed-sensor pulses
USART2	TX/RX	PA2/PA3	Telemetry to Raspberry Pi
GPIO out	—	PA6, PC6	L298N IN1 / IN2 (direction)

The three-ADC split is deliberate: throttle and temperature share ADC1's scan sequence because they tolerate being read back-to-back; the current sensor gets a dedicated ADC2 conversion with the longest available sampling time (480 cycles) because its source impedance is raised by the external divider; and the brake pedal sits alone on ADC3 so that a brake read can never be delayed by another channel in a sequence.

Chapter 2

System Overview

2.1 The driver's view — user interface

The driver does not care about ADC ranks or H-bridge truth tables. From the seat, the system exposes exactly four groups of controls and one display:

1. **Gear selector** — four momentary buttons: Park, Reverse, Neutral, Drive (PRND).
2. **Drive-mode buttons** — Eco, City, Sport. Each selects a different accelerator-pedal-to-power mapping (the three transfer curves sketched in Figure 2.1: a capped, gentle curve for Eco; a linear map for City; a steep early-saturating map for Sport).
3. **Pedals** — accelerator and brake. The brake always wins.
4. **Turn-signal buttons** — left and right, with auto-cancel.
5. **HMI dashboard** — a display fed by a Raspberry Pi showing speed, gear, mode, motor temperature, current and state of charge.

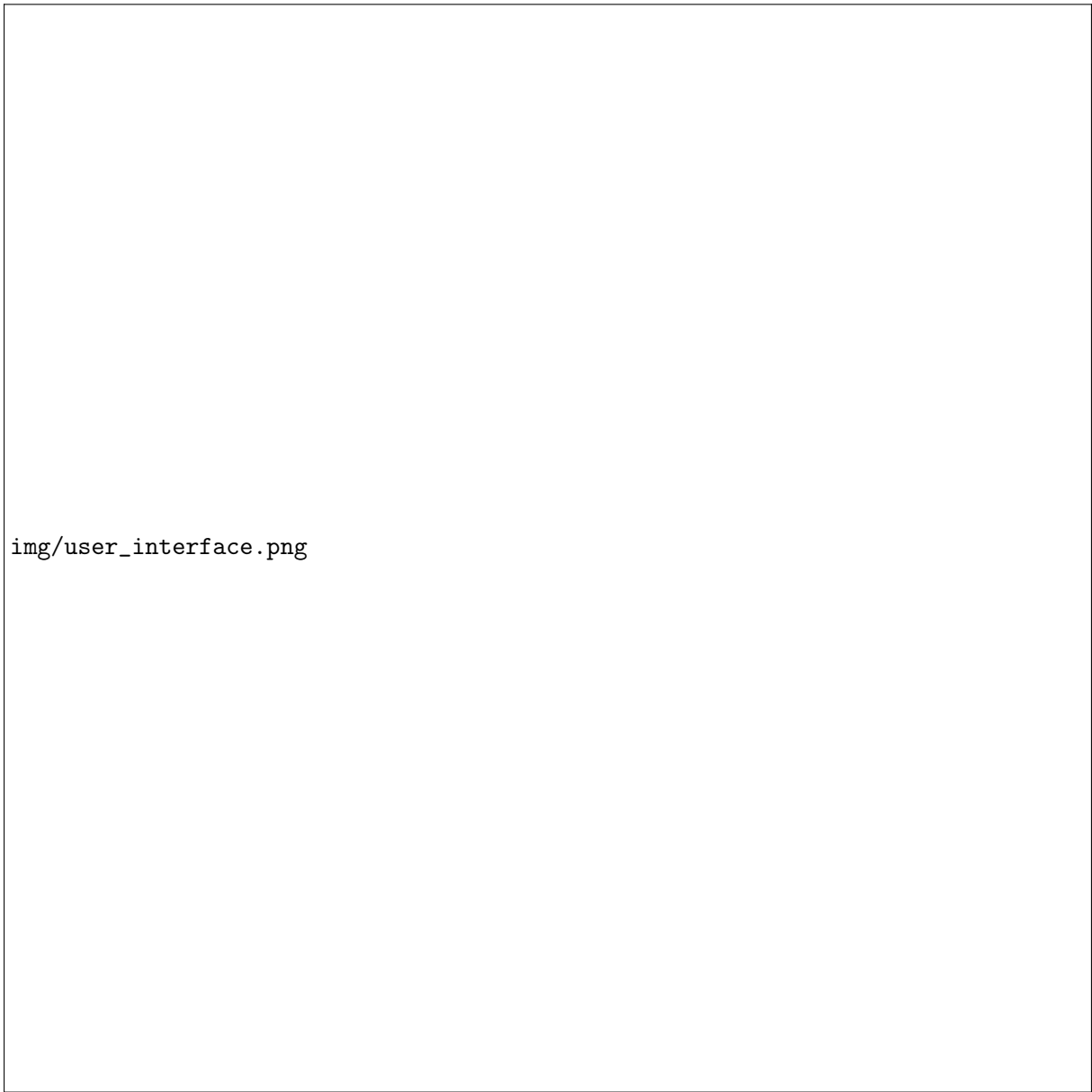
2.2 The engineer's view — hardware block diagram

Three architectural decisions in Figure 2.2 deserve a sentence each:

Series current sensing at the source. The ACS712 sits between the battery and the motor driver, not between driver and motor. This means the measurement includes the L298N's own quiescent draw, but it is the correct place for Coulomb counting: state of charge must account for every coulomb that leaves the battery, not only those that reach the motor windings.

Interrupts only where latency matters. Everything analog is polled inside a deterministic 100 Hz loop. Only the speed sensor uses an interrupt, because its pulses are asynchronous, short, and missing one directly corrupts the RPM figure. The mode switches are deliberately *not* on interrupts: mechanical bounce on an EXTI line generates interrupt storms, whereas polled debouncing costs nothing at 100 Hz.

Split grounds. Analog sensor returns and digital/switching returns are kept as separate nets and joined once. The L298N switches amperes at 1 kHz; letting that return current share a trace with a 10 k Ω NTC divider would modulate the temperature reading.

The image shows a user-facing view of a system dashboard. It features a large central display area with a light blue background. On the left side, there is a vertical sidebar containing several colored buttons (red, yellow, green, blue, orange) and a gear selector. The main display area contains three inset plots at the bottom, showing throttle transfer curves for Eco, Sport, and City modes. The text 'img/user_interface.png' is overlaid on the left side of the dashboard.

img/user_interface.png

Figure 2.1: User-facing view of the system. Driver inputs (gear selector, drive-mode buttons, pedals) converge on the STM32F446RE, which streams telemetry text to a Raspberry Pi driving the HMI dashboard. The three inset plots are the Eco, Sport and City throttle transfer curves.

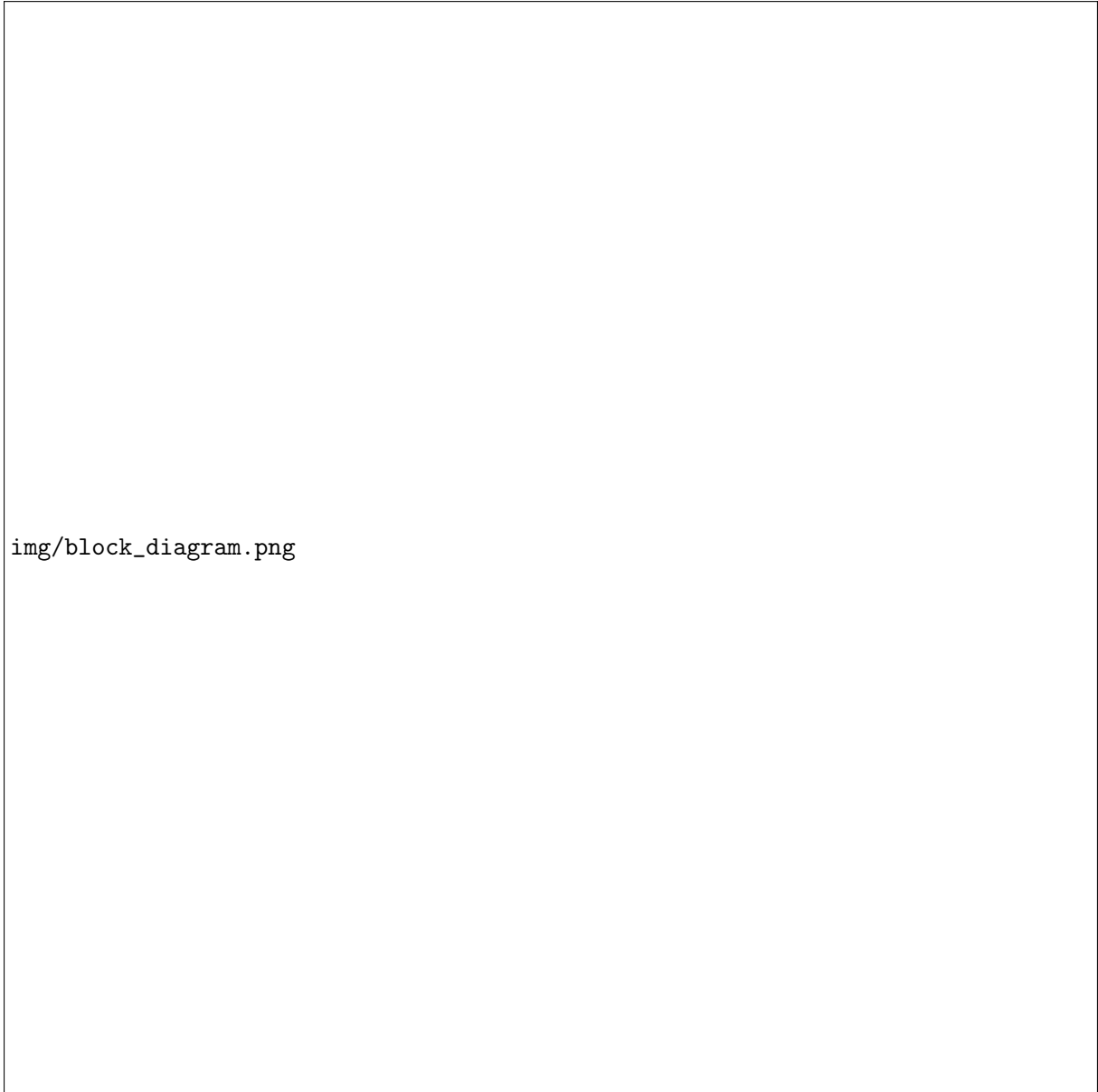


Figure 2.2: Hardware block diagram. A 12 V battery feeds the L298N motor driver through the ACS712 current sensor, so every ampere the motor draws is measured. The STM32F446RE reads the accelerator pedal, brake pedal, motor NTC and (divided) current-sensor output by ADC polling, counts speed-sensor pulses by external interrupt, debounces the mode switches in software, and commands the motor with a PWM output plus two direction lines. Analog and digital grounds are drawn separately and joined at a single point.

Chapter 3

Hardware Modules

3.1 STM32F446RE — the VCU brain

An ARM Cortex-M4F microcontroller (180 MHz capable, run here at 84 MHz from the internal HSI through the PLL) on the NUCLEO-F446RE board. The features this project actually leans on: three independent 12-bit ADCs (so the brake pedal never queues behind another conversion), general-purpose timers with PWM output, EXTI external-interrupt lines on every GPIO, and USART2 wired through the on-board ST-LINK virtual COM port — which is what carries the telemetry stream out.

3.2 ACS712-05B current sensor

A fully integrated Hall-effect linear current sensor (Allegro Microsystems). The load current passes through an internal 1.2 mΩ copper conduction path; the Hall element measures the resulting field, giving galvanic isolation between the power path and the signal side. Key datasheet figures for the -05B variant: optimised range ± 5 A, sensitivity 185 mV/A, 80 kHz bandwidth, total output error $\pm 1.5\%$ at 25 °C, single 5.0 V supply, ratiometric output centred at $V_{CC}/2 = 2.5$ V for zero current.

Two properties drive the interface design in Section 4.3. First, the output is *ratiometric*: both the zero-current midpoint and the sensitivity scale with the 5 V supply, so supply accuracy directly becomes measurement accuracy. Second, the output swings around 2.5 V on a 5 V scale, while the STM32's ADC reference is 3.3 V — hence the mandatory resistive divider.

The -05B was chosen over the 20A/30A variants because the bench motor draws well under 5 A, and the 05B's 185 mV/A sensitivity is almost triple that of the 30A part — resolution is spent where the signal actually lives.

3.3 KY-013 analog temperature sensor

A thermistor breakout: a 10 kΩ NTC (negative temperature coefficient) thermistor with a nominal Beta of about 3950, arranged with a fixed 10 kΩ resistor as a voltage divider. The output is an analog voltage that moves with temperature. It is mounted against the motor casing to track motor temperature.

An NTC is nonlinear by nature; the firmware linearises it with the simplified Steinhart–Hart (Beta) equation rather than a lookup table — one $\log f()$ call per second-scale reading is cheap on a Cortex-M4F with an FPU, and the Beta model is accurate to roughly ± 1 °C over the 0 °C to 100 °C span this application cares about.

3.4 L298N motor driver

A dual full-bridge (H-bridge) driver from STMicroelectronics: operating supply up to 46 V, 2 A DC per bridge (4 A total), TTL-compatible logic inputs, and built-in overtemperature protection. One bridge is used. Its three control lines are the whole motor interface: IN1/IN2 select direction or braking per the truth table in Section 5.4, and ENA gates the bridge — driving ENA with PWM modulates the average motor voltage and therefore speed.

Two honest limitations, both accepted for a bench POC: the L298N is a BJT design, so it drops roughly 1.8 V to 4.9 V total across the bridge depending on load current — a real efficiency loss from a 12 V supply — and the bare IC has no freewheeling diodes, so the (module's) external fast diodes are

load-bearing safety components, not optional extras. The datasheet rates commutation up to 25 kHz to 40 kHz; the chosen 1 kHz PWM is comfortably inside that.

3.5 Johnson 12 V DC motor

A brushed 12 V DC geared motor (“Johnson” type) acting as the traction motor of the model. Brushed DC was chosen deliberately: speed is proportional to average applied voltage, torque to current, so a single PWM channel plus an H-bridge gives full four-quadrant-style control without commutation electronics. Its stall/startup current must stay inside the L298N’s 2 A continuous rating; the ACS712 reading doubles as the check on that.

3.6 Slotted optical speed sensor (LM393 module)

An FC-03-style slotted opto-interrupter: an IR LED faces a phototransistor across a slot, and an LM393 comparator squares the result into a clean digital edge. The output stage is open-collector, which has a direct firmware consequence: the line needs a pull-up, and the module’s drive strength informed the decision to fit an external 10 k Ω pull-up rather than rely on the MCU’s weak internal one (see Section 4.5). A one-slot disc on the motor shaft interrupts the beam once per revolution.

Chapter 4

Interfacing: Connections and Pseudo Code

Every section below follows the same pattern: the physical connection, why it is wired that way, and pseudo code that mirrors the implemented firmware. The pseudo code uses the numbered-comment style of the source.

4.1 Accelerator pedal

Hardware connection

The accelerator pedal is modelled by a potentiometer used as a voltage divider:

Pot terminal	Connects to	Note
End A	3.3 V (MCU analog rail)	Never 5 V — the wiper feeds an ADC pin.
End B	Analog ground	
Wiper	PA0 (ADC1 IN0, rank 1)	0 V = pedal released, 3.3 V = floored.

ADC1 runs in scan mode with two ranks (throttle then NTC) and a 144-cycle sampling time. The long sampling time matters: a potentiometer mid-travel presents up to a quarter of its total resistance as source impedance, and the ADC's sample-and-hold capacitor needs time to charge through it. At the configured ADC clock ($84\text{ MHz}/4 = 21\text{ MHz}$), $144 + 12$ cycles $\approx 7.4\text{ }\mu\text{s}$ per conversion — irrelevant at a 100 Hz loop rate.

Pseudo code — throttle acquisition and mapping

```
1  /* 1 // start ADC1 scan sequence (rank1 = throttle, rank2 = NTC) */
2  HAL_ADC_Start(&hadc1);
3
4  /* 2 // poll rank 1: raw pedal position, 0..4095 */
5  if (poll_ok(&hadc1))
6      raw_throttle_adc = HAL_ADC_GetValue(&hadc1);
7
8  /* 3 // deadband: ignore noise at the pedal floor */
9  if (raw_throttle_adc < 50)
10     target_pwm = 0;
11
12 /* 4 // map raw ADC -> PWM % according to the active drive mode */
13 switch (current_drive_mode)
14 {
15     case MODE_ECO: /* gentle, capped at 60% */
16         target_pwm = (raw_throttle_adc * 60) / 4095; break;
17
18     case MODE_CITY: /* linear 1:1, up to 99% */
19         target_pwm = (raw_throttle_adc * 99) / 4095; break;
20
21     case MODE_SPORT: /* aggressive: full power at ~61% pedal travel */
22         target_pwm = (raw_throttle_adc * 99) / 2500;
23         if (target_pwm > 99) target_pwm = 99; break;
24 }
```

Listing 4.1: Accelerator pedal read and drive-mode mapping.

The three maps are the product differentiator of the drive modes. Eco trades top-end power (hard cap at 60 % duty) for range; City is the predictable linear reference; Sport reaches 99 % duty at an ADC count of only ~ 2500 (≈ 61 % pedal travel), which *feels* faster because the same foot input produces more torque earlier.

4.2 Brake pedal

Hardware connection

Identical topology to the accelerator — a potentiometer between 3.3 V and analog ground with the wiper on PC2 — but on its own converter, ADC3 channel 12, single conversion, 144-cycle sampling. Giving the brake a dedicated ADC is a safety-shaped decision: the brake read is the first acquisition of every control cycle and can never be queued behind, or corrupted by, another channel’s conversion.

Pseudo code — brake acquisition and override

```
1  /* 1 // read the brake pot on its dedicated converter (ADC3 CH12) */
2  HAL_ADC_Start(&hadc3);
3  if (poll_ok(&hadc3))
4      raw_brake_adc = HAL_ADC_GetValue(&hadc3);
5  HAL_ADC_Stop(&hadc3);
6
7  /* 2 // convert to percent with a floor deadband */
8  brake_percent = (raw_brake_adc < 50) ? 0
9                  : (raw_brake_adc * 100) / 4095;
10
11 /* 3 // PRIORITY 1 of the safety chain: brake always wins.
12    Threshold of 5% needs calibration on the bench pot. */
13 if (brake_percent > 5)
14     throttle_percent = 0;
15 /* do NOT return: FSM, SoC and debouncing must still run */
```

Listing 4.2: Brake pedal read; brake is priority 1 of the safety chain.

Note the comment in step 3: the override zeroes the throttle and *falls through*. Cutting the loop short with an early return would freeze Coulomb counting and gear debouncing while braking — a subtle class of bug this structure avoids by design.

4.3 Current sensor (ACS712-05B)

Hardware connection

ACS712 module pin	Connects to	Note
IP+ / IP-	In series: battery + $\rightarrow$ L298N V_S	The measured power path.
VCC	5 V	Sensor requires 5 V; output is ratiometric to it.
GND	Analog ground	
VIOUT	Divider top	See below.
Divider node	PA1 (ADC2 IN1)	480-cycle sampling time.

The level-shifting divider. The sensor output is centred at 2.5 V and can approach 5 V, but the STM32's analog reference is 3.3 V. A resistive divider with $R_{top} = 2.0\text{k}\Omega$ (from VIOUT) and $R_{bot} = 4.3\text{k}\Omega$ (to analog ground) scales the signal:

$$k = \frac{R_{bot}}{R_{top} + R_{bot}} = \frac{4.3}{2.0 + 4.3} \approx 0.683 \quad (4.1)$$

From this ratio the two calibration constants follow:

$$V'_{mid} = 2.5\text{ V} \times 0.683 = 1.706\text{ V} \quad \Rightarrow \quad \text{ZERO_OFFSET} = \frac{1.706}{3.3} \times 4095 \approx 2117 \quad (4.2)$$

$$S' = 185\text{ mV/A} \times 0.683 = 126.3\text{ mV/A} \quad \Rightarrow \quad \text{COUNTS_PER_AMP} = \frac{126.3}{3300} \times 4095 \approx 157 \quad (4.3)$$

The firmware uses $\text{ZERO_OFFSET} = 2100$ and $\text{COUNTS_PER_AMP} = 152$ — within about 1–3 % of the theoretical values, consistent with empirical trimming against real resistor tolerances and the actual 5 V rail. *These constants are functions of the divider; change either resistor and both must be recalculated.* (An earlier in-code comment derives 1861/138 from a 10 k Ω /15 k Ω divider that is not the fitted hardware.)

Why the longest sampling time. ADC2 samples this channel for 480 cycles ($\approx 23\text{ }\mu\text{s}$), the maximum available. The divider's Thévenin source impedance ($R_{top} \parallel R_{bot} \approx 1.4\text{k}\Omega$) is modest, but the current signal is the one input where a few counts of error integrate forever (into state of charge), so the cheapest accuracy available — sampling time — is spent here.

Pseudo code — current calculation and Coulomb increment

```

1  /* 1 // read the ADC value from the current sensor (ADC2 CH1, PA1) */
2  HAL_ADC_Start(&hadc2);
3  if (poll_ok(&hadc2))
4      raw_current_adc = HAL_ADC_GetValue(&hadc2);
5  HAL_ADC_Stop(&hadc2);
6
7  /* 2 // convert ADC value to current:
8      ZERO_OFFSET      = count at 0 A (midpoint after divider) = 2100
9      COUNTS_PER_AMP  = counts per ampere after divider      = 152
10     */
11  motor_current_amps =
12      ((float)raw_current_adc - ZERO_OFFSET) / COUNTS_PER_AMP;
13
14  /* 3 // deadband: even at 0 rpm the reading jitters around zero.
15     Negative values are expected because of the offset subtraction.
16     */
17  if (motor_current_amps > -0.20f && motor_current_amps < 0.20f)
18      motor_current_amps = 0.0f;
19
20  /* 4 // Coulomb counting: integrate only real discharge current.
21     delta_hours = delta_ms / 3,600,000 (ms -> hours)
22     delta_ms    = now - last_vcu_tick (last loop execution) */
23  if (motor_current_amps > 0.0f)
24      capacity_consumed_ah += motor_current_amps * delta_hours;

```

Listing 4.3: ACS712 read, conversion, deadband, and charge integration.

The deadband ($\pm 0.20\text{ A}$) is not cosmetic. Without it, a standing offset of even half a percent would be integrated by step 4 for hours and silently drain the *computed* state of charge of a battery that is physically full. The band is sized above the sensor's noise floor plus residual offset error after calibration.

4.4 Analog temperature sensor (KY-013)

Hardware connection

Element	Connects to	Note
Fixed 10 kΩ	3.3 V → divider node	Series (top) resistor.
NTC 10 kΩ	Divider node → analog ground	Low-side thermistor.
Divider node	PC1 (ADC1 IN11, rank 2)	144-cycle sampling.

With the NTC on the low side, the node voltage *falls* as temperature rises (NTC resistance drops). The firmware first inverts the divider to recover the thermistor’s resistance, then applies the Beta equation:

$$R_{NTC} = R_{series} \cdot \frac{N_{raw}}{4095 - N_{raw}} \quad \frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \ln \frac{R_{NTC}}{R_0} \quad (4.4)$$

with $T_0 = 298.15$ K, $R_0 = 10$ kΩ, $B = 3950$.

Pseudo code — temperature and thermal thresholds

```
1  /* 1 // raw ADC (ADC1 rank 2) -- clamp to avoid divide-by-zero/log(0)
   */
2  if (raw == 0)    raw = 1;
3  if (raw >= 4095) raw = 4094;
4
5  /* 2 // invert the divider: ADC count -> NTC resistance */
6  r_ntc = NTC_SERIES_R * ((float)raw / (4095.0f - (float)raw));
7
8  /* 3 // Beta equation: resistance -> kelvin -> celsius */
9  temp_K = 1.0f / ( (1.0f/298.15f) + (1.0f/3950.0f)*logf(r_ntc/10000.0f
   ) );
10 motor_temp_C = temp_K - 273.15f;
11
12 /* 4 // two-level protection, evaluated in the safety chain:
13    >= 65 C : FAULT -> throttle forced to 0 (priority 2)
14    >= 50 C : WARNING -> throttle capped at 50% (priority 4) */
15 if (motor_temp_C >= TEMP_FAULT_C)
16     throttle_percent = 0;
17 else if (motor_temp_C >= TEMP_WARNING_C && throttle_percent > 50)
18     throttle_percent = 50;
```

Listing 4.4: NTC conversion and the two-level thermal derate.

The clamping in step 1 is defensive numerics: a raw reading of exactly 0 or 4095 (disconnected or shorted sensor) would otherwise produce a division by zero or $\ln(0)$. A disconnected low-side NTC reads near full scale, which the Beta equation maps to an implausibly high resistance and hence a very *low* temperature — a known blind spot in this sensing approach.

4.5 Speed sensor

Hardware connection

Module pin	Connects to	Note
VCC	3.3 V	Keeps the output swing ADC/GPIO safe.
GND	Digital ground	
OUT	PC0 + external 10 kΩ pull-up to 3.3 V	Open-collector output.

PC0 is configured as EXTI line 0. The comparator output is open-collector: it can only pull the line low, so the high level comes entirely from the pull-up. An external 10 k Ω resistor was fitted because the MCU's internal pull-up (nominally 30 k Ω to 50 k Ω) gives slow rising edges on a wired line — slow edges near the input threshold can double-count.

Signal convention. Slot clear = LOW; disc segment blocking the slot = HIGH. With a one-slot disc, one rising edge = one revolution, so PULSES_PER_REV = 1.

Pseudo code — pulse counting and RPM window

```

1  /* --- interrupt context (EXTI0) ----- */
2  /* 1 // count only rising edges: pin HIGH = slot just blocked */
3  void HAL_GPIO_EXTI_Callback(uint16_t pin)
4  {
5      if (pin == Speed_sensor_Pin &&
6          HAL_GPIO_ReadPin(GPIOC, Speed_sensor_Pin) == GPIO_PIN_SET)
7          ir_pulse_count++;    /* volatile: shared with main loop */
8  }
9
10 /* --- main loop, every RPM_WINDOW_MS = 1000 ms -----
11    */
12 /* 2 // atomically take-and-reset the shared counter */
13 __disable_irq();
14 pulses = ir_pulse_count;
15 ir_pulse_count = 0;
16 __enable_irq();
17
18 /* 3 // the "holy grail" formula:
19    RPM = (pulses / PULSES_PER_REV) / elapsed_s * 60
20    = (pulses * 60000) / (PULSES_PER_REV * elapsed_ms)    */
21 motor_rpm = (pulses * 60000.0f) / (PULSES_PER_REV * elapsed_ms);

```

Listing 4.5: ISR pulse counter and once-per-second RPM computation.

Two details are load-bearing. The take-and-reset in step 2 happens with interrupts briefly masked because a 32-bit read-modify-write racing an ISR increment can lose pulses. And the 1 s window is a resolution decision: with one pulse per revolution, a 1 s window quantises RPM in steps of 60 — already coarse — while a shorter window would make low speeds unreadable. Calibration guidance: if the disc is changed to N slots, set PULSES_PER_REV = N and verify against a handheld tachometer.

4.6 Switch debouncing

Hardware connection

All nine command buttons (four gears, three modes, two blinkers) share one topology: momentary switch from the pin to ground, MCU internal pull-up enabled, active LOW. No RC network is fitted — debouncing is purely in software, which keeps the BOM at zero and the timing tunable.

Button	Pin	Button	Pin
Gear Park	PB1	Mode Eco	PB10
Gear Neutral	PA9	Mode City	PB4
Gear Drive	PB7	Mode Sport	PB5
Gear Reverse	PB2	Blinker L / R	PB12 / PB13

Debounce logic

Each button owns a small state structure (`Button_t`) holding its raw state, stable state, the timestamp of the last falling edge, and a one-shot `event_fired` flag. The rule: a press only becomes an *event* after the pin has read LOW continuously for `DEBOUNCE_MS = 30 ms` (three consecutive loop iterations).

```
1  /* 1 // sample the pin (active LOW: 0 = pressed) */
2  read = (HAL_GPIO_ReadPin(btn->port, btn->pin) == RESET) ? 0 : 1;
3
4  if (read == 0)                                /* being held down */
5  {
6      if (btn->raw_state == 1)                    /* 2 // falling edge: */
7      {                                           /* start the timer */
8          btn->raw_state = 0;
9          btn->press_tick = HAL_GetTick();
10     }
11     else if (!btn->event_fired &&
12              HAL_GetTick() - btn->press_tick >= DEBOUNCE_MS)
13     {
14         btn->stable_state = 0;                  /* 3 // held for 30 ms: */
15         btn->event_fired = 1;                  /* fire ONE event */
16     }
17 }
18 else                                           /* 4 // released: reset */
19 {
20     btn->raw_state = 1;
21     btn->stable_state = 1;
22     btn->event_fired = 0;
23 }
24
25 /* 5 // consumer pattern: whoever reacts to the event clears it */
26 if (btn_drive.event_fired) { btn_drive.event_fired = 0; ... }
```

Listing 4.6: Per-button debounce state machine, run every 10 ms.

Why 30 ms: typical tactile-switch bounce settles within 1 ms to 10 ms; 30 gives a $3\times$ margin while staying far below the ~ 100 ms threshold where a human starts perceiving lag. Why polled, not EXTI: nine bouncing contacts on interrupt lines would generate bursts of spurious interrupts precisely when timing matters; polling at 100 Hz converts the problem into three cheap comparisons per button per cycle.

Chapter 5

Control Logic

5.1 The 100 Hz VCU loop

All control runs in one superloop clocked by SysTick comparison, not by blocking delays:

```
1  if (now - last_vcu_tick >= VCU_LOOP_MS)      /* every 10 ms */
2  {
3      A. read brake pedal          (ADC3)
4      B. read throttle + NTC      (ADC1 scan)
5      C. raw counts -> percent
6      D. SAFETY PRIORITY CHAIN    (brake > fault > mapping > warning)
7      E. debounce all 9 buttons; apply mode switches
8      F. gear FSM transitions     (with interlocks)
9      G. apply motor state        (IN1/IN2 + PWM compare)
10     H. blinker logic
11     I. non-blocking warning LED
12     J. read current sensor      (ADC2)
13     K. Coulomb counting -> SoC
14 }
```

Listing 5.1: Top-level loop structure (one iteration).

5.2 Safety priority chain

Ordering is the safety argument, so it is explicit and fixed:

1. **Brake** ($\text{brake_percent} > 5$) — throttle forced to zero. A mechanical-analog input outranks every computed value.
2. **Thermal fault** ($\geq 65^\circ\text{C}$) — full torque cutoff.
3. **Normal mapping** — the drive-mode transfer curve.
4. **Thermal warning** ($\geq 50^\circ\text{C}$) — applied *after* mapping as an unconditional 50 % cap, so no mode can out-vote a hot motor.

Every level zeroes or caps the throttle and *falls through*; nothing early-returns. Debouncing, the FSM and Coulomb counting execute on every iteration regardless of fault state — a driver must be able to shift to Park during a thermal fault.

5.3 Gear FSM (PRND)

From	Allowed to	Blocked	Interlock
PARK	N, D, R	—	—
NEUTRAL	P, D, R	—	N→R only if $\text{RPM} < 150$
DRIVE	N, P	R	“money shift” $\text{D} \rightarrow \text{R} \Rightarrow$ 3-flash warning
REVERSE	N, P	D	$\text{R} \rightarrow \text{D}$ likewise blocked

Direct D↔R transitions are refused outright: reversing an H-bridge under way is both a mechanical shock and, on the electrical side, a plugging event where back-EMF adds to the supply across the bridge. The required sequence is D→N→R, and even N→R is gated by the measured RPM. A blocked attempt triggers a non-blocking three-flash warning on LD2 (six 250 ms toggles) rather than a HAL_Delay — the control loop never stalls.

5.4 Motor output stage

Gear state	IN1 (PA6)	IN2 (PC6)	ENA (TIM2 CH1)
PARK	LOW	LOW	0 — both low-side devices on: motor terminals shorted (hard brake)
NEUTRAL	LOW	LOW	0 — coast
DRIVE	HIGH	LOW	throttle_percent (0–99)
REVERSE	LOW	HIGH	throttle_percent, hard-capped at 30

The 30 % reverse cap is a policy decision borrowed from real EVs: reverse exists for manoeuvring, not for speed. PWM parameters: TIM2 at 84 MHz, prescaler 833, ARR 99 $\Rightarrow f_{PWM} = 84\text{ MHz}/(834 \times 100) \approx 1.007\text{ kHz}$ with exactly 1 % duty resolution — throttle_percent writes into the compare register with no scaling.

5.5 State of charge (Coulomb counting)

$$SoC = 100\% \times \left(1 - \frac{\int i dt}{C_{batt}}\right), \quad C_{batt} = 4\text{ Ah} \quad (5.1)$$

Each 10 ms cycle adds $i \times \Delta t_{hours}$ to capacity_consumed_ah whenever the deadbanded current is positive. The integrator is clamped so SoC never goes below zero. Known model limits (accepted for a POC): no charge accounting, no rest-voltage correction, and any residual sensor offset above the deadband integrates as phantom drain.

5.6 Turn signals

Left/right buttons toggle their own side (press again to cancel); selecting one side replaces the other; and an auto-cancel extinguishes the signal after 3 s. The LEDs on PB14/PB15 are driven every loop from blinker_state — the state machine, not the ISR-style flags, owns the outputs.

Chapter 6

Telemetry and HMI

6.1 UART Link to Raspberry Pi

6.2 Telemetry Frame Format

6.3 Raspberry Pi Receiver

6.4 HMI Dashboard

6.5 Status and Open Items

Appendix A

Complete Pin Map (from the CubeMX .ioc)

Pin	Label	Function	Electrical
PA0	acc_pedal	ADC1 IN0	Analog in
PA1	CUrrent_sensor	ADC2 IN1	Analog in
PA2	USART_TX	USART2 TX	AF, to ST-LINK VCP / Pi
PA3	USART_RX	USART2 RX	AF
PA5	LD2	Warning LED	Push-pull out
PA6	forward_driver	L298N IN1	Push-pull out
PA8	—	GPIO input (spare)	No pull
PA9	gear_neutral	Gear N button	Input, pull-up
PA15	pwm	TIM2 CH1	PWM to L298N ENA
PB1	gear_park	Gear P button	Input, pull-up
PB2	gear_reverse	Gear R button	Input, pull-up
PB4	btn_city	Mode City button	Input, pull-up
PB5	btn_sport	Mode Sport button	Input, pull-up
PB7	gear_drive	Gear D button	Input, pull-up
PB10	btn_eco	Mode Eco button	Input, pull-up
PB12	btn_blinker_left	Blinker L button	Input, pull-up
PB13	btn_blinker_right	Blinker R button	Input, pull-up
PB14	led_blinker_left	Blinker L LED	Push-pull out
PB15	led_blinker_right	Blinker R LED	Push-pull out
PC0	Speed_sensor	EXTI0	Input, rising+falling IT
PC1	temp_motor	ADC1 IN11	Analog in
PC2	break_pedal	ADC3 IN12	Analog in
PC6	reverse_driver	L298N IN2	Push-pull out
PC13	B1	Blue user button	EXTI13 (board)
PA13/PA14/PB3	TMS/TCK/SWO	SWD debug	Reserved

Clock configuration: SYSCLK = 84 MHz from HSI via PLL (M=16, N=336, P=4); APB1 at 42 MHz (timers $\times 2 = 84$ MHz); ADC clock = PCLK2/4 = 21 MHz. A DMA2 Stream 0 circular channel is provisioned for ADC1 in the .ioc but the firmware currently polls (noted in the source header: “NOT IMPLEMENTED DMA FOR ADC1”).